

操作计划追踪系统 — 设计文档 v2

1. 背景与目标

工作台(Workbench)依赖用户手动填写“明日计划”，数据源经过人工筛选，不适合追踪系统本身的判断质量。

v2 目标： 复盘页面每天17:00实时计算的 `trading_plan`（每日交易计划）中的个股操作数据，自动追踪次日涨跌表现。

核心问题

- 系统说“中继买点” → 第二天实际涨了还是跌了？各买点类型成功率多少？
- 系统说“上涨趋势·上行” → 这个结构+阶段判断准不准？
- 主线板块的股票 vs 非主线，成功率差别多大？
- 持仓操作建议的准确率 vs 关注买入建议的准确率？

2. 数据源

数据来源

不再依赖工作台文件（`workbench/{date}.json`），改为直接从复盘实时计算结果中提取：

```
from backend.services.review_service import compute_review_real_time
data = compute_review_real_time(date_str)
trading_plan = data['trading_plan']
```

`trading_plan` 包含两个个股操作列表：

holdings_action[] — 持仓股操作建议 | 字段 | 类型 | 示例 | |—| |—| |—| | `stock` | `str` | “国际复材(301526)” | `action` | `str` | “执行突破买点” / “持有不动” / “卖出” | `reason` | `str` | “上涨趋势·上行” (`structure·stage`) | `priority` | `str` | “高” / “中” / “低” | `stop_loss` | `float` | 止损价 (如果有) | `stop_loss_pct` | `float` | 止损百分比 | `change` | `float` | 当日涨跌幅 |

buy_priority[] — 关注买入信号 | 字段 | 类型 | 示例 | |—| |—| |—| | `name` | `str` | “广钢气体” | `code` | `str` | “688548” | `sector` | `str` | “电子化学品” | `buy_point` | `str` | “中继买点” / “涨停回踩” / “趋势回踩EMA5” | `is_main` | `bool` | 是否主线板块 | `profit_model1` | `bool` | 是否盈利模式1 | `trend_stock` | `bool` | 是否趋势股 | `structure` | `str` | “上涨趋势” | `stage` | `str` | “上行” | `change` | `float` | 当日涨跌幅 | `stop_loss` | `float` | 止损价 | `stop_loss_pct` | `float` | 止损百分比 | `priority` | `int` | 优先级排序 |

注意：数据源是实时计算的 `compute_review_real_time(date_str)`，不依赖任何存档文件。

3. 数据模型与存储

存储方案：SQLite

使用 Python 标准库 `sqlite3`，零外部依赖，适合多维度聚合统计。

数据库路径： `{DATA_DIR}/private/plan_tracking.db`

表结构

```
CREATE TABLE IF NOT EXISTS plan_records (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  date        TEXT      NOT NULL,      -- 计划日期 '2026-05-28'
  code        TEXT      NOT NULL,      -- 股票代码
  name        TEXT,        -- 股票名称

  -- 来源分类
  source       TEXT,      -- 'holdings_action' 或 'buy_priority'
  action       TEXT,      -- 系统建议操作
  reason       TEXT,      -- 结构·阶段：'上涨趋势·上行'
  structure    TEXT,      -- 结构：上涨趋势/区间震荡/下降趋势
  stage        TEXT,      -- 阶段：上行/缩量整理/区底
  buy_point    TEXT,      -- 买点类型
  is_main      INTEGER,   -- 0/1 是否主线
```

```

    priority          TEXT,      -- 高/中/低

-- 止损
stop_loss            REAL,      -- 止损价
stop_loss_pct        REAL,      -- 止损百分比

-- 计划当天行情
plan_close           REAL,      -- 当日收盘价

-- 次日追踪
next_date            TEXT,      -- 下一个交易日
next_open            REAL,      -- 次日开盘
next_close           REAL,      -- 次日收盘
next_high            REAL,      -- 次日最高
next_low             REAL,      -- 次日最低

-- 计算结果
change_pct           REAL,      -- 次日涨跌幅
max_gain             REAL,      -- 盘中最大涨幅
max_loss             REAL,      -- 盘中最大跌幅
hit_stop_loss        INTEGER,   -- 0/1 盘中是否跌破止损
result               TEXT,      -- success / failure / flat / pending / no_data

-- 用户标记
executed             INTEGER,   -- NULL=未知 0=未执行 1=已执行
user_note            TEXT,      -- 用户备注

-- 时间戳
created_at           TEXT,      -- 首次写入时间
updated_at           TEXT,      -- 最后更新时间

    UNIQUE(date, code)
);

-- 统计维度的索引
CREATE INDEX idx_records_date      ON plan_records(date);
CREATE INDEX idx_records_result    ON plan_records(result);
CREATE INDEX idx_records_source    ON plan_records(source);
CREATE INDEX idx_records_buy_point ON plan_records(buy_point);
CREATE INDEX idx_records_structure ON plan_records(structure, stage);
CREATE INDEX idx_records_is_main   ON plan_records(is_main);

```

多维度统计

```

-- 成功率统计（任意维度组合）
SELECT buy_point, count(*) as total,
       sum(CASE WHEN result='success' THEN 1 ELSE 0 END) as wins,
       round(avg(change_pct), 2) as avg_chg
FROM plan_records
WHERE result IN ('success','failure')
GROUP BY buy_point;

-- 按结构·阶段
SELECT structure, stage, count(*), avg(change_pct)
FROM plan_records GROUP BY structure, stage;

-- 按是否主线
SELECT is_main, count(*) as total,
       sum(CASE WHEN result='success' THEN 1 ELSE 0 END) as wins,
       round(avg(CASE WHEN result='success' THEN 1.0 ELSE 0.0 END) * 100, 1) as rate
FROM plan_records WHERE result IN ('success','failure')
GROUP BY is_main;

-- 交叉聚合：主线+买点
SELECT is_main, buy_point, count(*) as total,
       sum(CASE WHEN result='success' THEN 1 ELSE 0 END) as wins
FROM plan_records WHERE result IN ('success','failure')
GROUP BY is_main, buy_point;

```

成功/失败判定规则

同 v1，保持不变：

买入方向（含 **buy_priority** + **holdings_action** 中买入建议）：| 条件 | 判定 | |——|——| | 次日收盘 > 当日收盘 +0.5% | ✓
success | | 次日收盘 < 当日收盘 -0.5% | ✗ failure | | 介于 ±0.5% | — flat | | 无次日数据 | ⌘ pending |

卖出方向：| 条件 | 判定 | |——|——| | 次日收盘 < 当日收盘 -0.5% | ✓ success（卖对了） | | 次日收盘 > 当日收盘 +0.5% | ✗ failure（卖飞了） |

持有/观察方向：不参与成功率统计，仅展示次日涨跌。

4. 系统架构



5. API 设计

GET /api/plan-tracking

```
{
  "plans": [
    {
      "id": 1,
      "date": "2026-05-28",
      "code": "301526",
      "name": "国际复材",
      "source": "holdings_action",
      "action": "执行突破买点",
      "reason": "上涨趋势·上行",
      "structure": "上涨趋势",
      "stage": "上行",
      "buy_point": "突破买点",
      "is_main": 1,
      "priority": "高",
      "plan_close": 12.50,
      "next_date": "2026-05-29",
      "next_close": 13.20,
      "change_pct": 5.6,
      "result": "success",
      "executed": null
    }
  ],
  "summary": {
```

```
    "total_plans": 22,
    "success": 14,
    "failure": 6,
    "flat": 2,
    "pending": 0,
    "success_rate": 70.0,
    "avg_gain_pct": 3.2,
    "avg_loss_pct": -2.1,
    "best_gain": 8.5,
    "worst_loss": -4.3,
    "win_loss_ratio": 1.52
  },
  "by_buy_point": {
    "中继买点": { "total": 8, "success": 6, "failure": 1, "flat": 1, "rate": 85.7 },
    "涨停回踩": { "total": 5, "success": 3, "failure": 2, "flat": 0, "rate": 60.0 },
    "趋势回踩EMA5": { "total": 4, "success": 2, "failure": 1, "flat": 1, "rate": 66.7 }
  },
  "by_structure": {
    "上涨趋势": { "total": 12, "success": 9, "failure": 2, "flat": 1, "rate": 81.8 },
    "区间震荡": { "total": 8, "success": 4, "failure": 3, "flat": 1, "rate": 57.1 }
  },
  "by_is_main": {
    "1": { "total": 10, "success": 8, "failure": 1, "flat": 1, "rate": 88.9 },
    "0": { "total": 12, "success": 6, "failure": 5, "flat": 1, "rate": 54.5 }
  },
  "by_source": {
    "holdings_action": { "total": 12, "success": 7, "failure": 4, "flat": 1 },
    "buy_priority": { "total": 10, "success": 7, "failure": 2, "flat": 1 }
  },
  "suggestions": [
    {
      "type": "warning",
      "dimension": "buy_point",
      "category": "缩量整理",
      "rate_current": 33.3,
      "count": 3,
      "message": "条件「缩量整理」3次仅33%成功率"
    }
  ],
  "last_updated": "2026-05-30T10:00:00"
}
```

POST /api/plan-tracking/annotate

标记计划执行状态。

POST /api/plan-tracking/refresh

强制重新计算未完成的追踪。

6. 前端页面

复用 v1 的 PlanTracking.tsx，适配新数据模型： - 统计卡片行（成功率/平均盈利/盈亏比）保持不变 - 按买点类型分组（替代原来的条件类型） - 按结构·阶段分组（新增） - 按是否主线分组（新增） - 自动建议逻辑更新为新维度

7. 归类维度总结

维度	来源字段	统计示例
买点类型	buy_point	中继买点 85%
结构	structure	上涨趋势 81%
结构+阶段	structure, stage	上涨趋势·上行 85%
是否主线	is_main	主线 89%
数据来源	source	持仓操作 vs 关注买入

操作方向	action	买入建议 70%
------	--------	----------

8. 不做 (v1 范围内)

- 第3天/第5天追踪 — v2 再扩展
- 自动标记“已执行” — 需交易系统接入
- 实盘成交价追踪 — 无券商API
- 板块/大盘同期对比 — v2

9. 文件清单

新增/修改:

docs/plan-tracking-design.md

— 本设计文档 (v2更新)

server/backend/services/plan_tracking_service.py

— 重写: 数据源从 workbench→review, 存储从 JSON→SQLite

server/backend/api/plan_tracking.py

— 适配新数据模型

server/backend/tests/test_plan_tracking.py

— 重写测试 (TDD)

数据:

data/private/plan_tracking.db

— SQLite数据库 (自动生成, 取代 plan_tracking.json)

10. 分支策略

- 分支: feature/plan-tracking
- TDD 开发
- 完成后 PR 合并 master